Artificial Intelligence

Assignment 4

Group: Ritchie

1. Eshwar Reddy Poreddy: 221100642 (eshwaraa07@uni-koblenz.de)

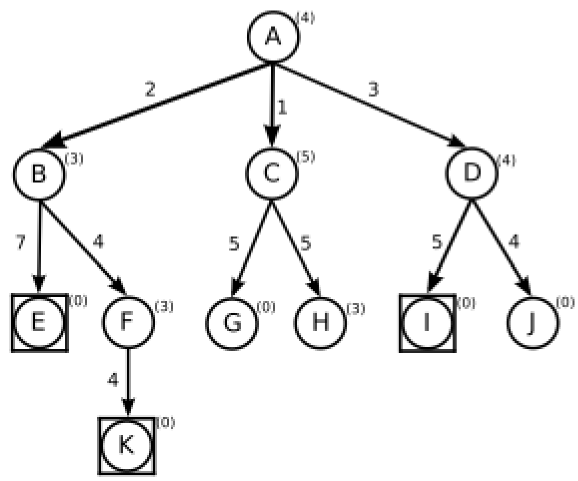
2. Prajna Shetty: 222100479 (prajnashetty73@uni-koblenz.de)

3. Vamshidhar Pratap: 222202552 (vamshidharpratap@uni-koblenz.de)

4. Yangsun Kim: 220202400 (ykim@uni-koblenz.de)

1 Uninformed and Informed Search (10 Points)

The following figure shows the complete search tree of a search problem. The start node is node A. all target nodes are marked with a square. The number next to each edge is its cost. A heuristic indicating the estimated distance to the closest target is shown in parentheses next to each node.



For each of the following search strategies, write down (in order) which nodes are visited

- Breadth first search
- Depth first search
- Iterative deepening with increment 1 (mark where the depth limit increases)
- A* If two nodes with the same value are present in the frontier, the one which was added to the frontier first will be selected. Provide the value for all nodes taken from the frontier
- A* but the heuristic for D is 8 instead of 4

Which solution is optimal? Why does one of the A-variants not find it?

Answers : -

Breadth First Search :

Goal Node	Order of Iteration	Cost Associated
E	A -> B -> C -> D -> E	2 + 1 + 3 + 7 = 13
K	A -> B -> C -> D -> E -> F -> G -> H -> I -> J -> K	2 + 1 + 3 + 7 + 4 + 5 + 5 + 5 + 4 + 4 = 40
I	A -> B -> C -> D -> E -> F -> G -> H -> I	2 + 1 + 3 + 7 + 4 + 5 + 5 + 5 = 32

Depth First Search :

Goal Node	Order of Iteration	Cost Associated
E	A -> B -> E	2 + 7 = 9
K	A -> B -> E -> F -> K	2 + 7 + 4 + 4 = 17
I	A -> B -> E -> F -> K -> C -> G -> H -> D -> I	2 + 7 + 4 + 4 + 1 + 5 + 5 + 3 + 5 = 34

ITERATIVE DEEPENING DEPTH FIRST SEARCH :

Goal Node	Order of Iteration	Cost Associated
E	Iteration 0 : A Iteration 1 : A -> B -> C -> D Iteration 2 : A -> B -> E	2+1+3+2+7
K	Iteration 0 : A Iteration 1 : A -> B -> C	2+1+2+7+4+1+5+5+ 3+5+4+2+7+4+4

	Iteration 2 : A -> B -> E -> F -> C -> G -> H -> D -> I -> J Iteration 3 : A -> B -> E -> F -> K	
I	Iteration 0 : A Iteration 1 : A -> B -> C Iteration 2 : A -> B -> E -> F -> C -> G -> H -> D -> I	2+1+2+7+4+5+5

A*

	F(n) = g(n) + h(n)	
A		Explored = { } Frontier = { A }
A -> B A -> C A -> D	2 + 3 = 5 1 + 5 = 6 3 + 4 = 7	min (5, 6, 8) = 5 Explored = { A , B } Frontier = { C, D }
A -> B -> E A -> B -> F	(2 + 7)+ 0 = 9 (2 + 4) + 3 = 9	min (6, 8, 9, 9) = 6 Explored = { A , B, C } Frontier = { D, E, F }
A -> C -> G A -> C -> H	(1 + 5) + 0 = 6 (1 + 5) + 3 = 9	min (8, 9, 9, 6, 9) = 6 Explored = { A , B, C, G } Frontier = { D, E, F , H}
		min (8, 9, 9, 9) = 8 Explored = { A , B, C, G, D } Frontier = { E, F , H}
A -> D -> I A -> D -> J	(3 + 5) + 0 = 8 (3 + 4) + 0 = 7	min (9, 9, 9, 8, 7) = 7 Explored = { A , B, C, G, D, J } Frontier = { E, F , H, I}
		min (9, 9, 9, 8) = 8

		Explored = { A , B, C, G, D, J, I } Frontier = { E, F , H}
		min (9, 9, 9) = 9 Explored = { A , B, C, G, D, J, I , E} Frontier = { F , H }
A -> K	2 + 4 + 4 + 0 = 10	min (9, 9, 10) = 9 Explored = { A , B, C, G, D, J, I , E, F} Frontier = { H , K }
		Explored = { A , B, C, G, D, J, I , E, F, H} Frontier = { K }
		Explored = { A , B, C, G, D, J, I , E, F, H, K} Frontier = { }

Goal Node	Order of Iteration	Cost Associated
E	A -> B -> C -> G -> D -> J -> I -> E	2 + 1 + 5 + 3 + 4 + 5 +7 = 27
K	A -> B -> C -> G -> D -> J -> I -> E -> F -> H -> K	2 + 1 + 5 + 3 + 4 + 5 +7 + 4 + 5 +4 = 40
I	A -> B -> C -> G -> D -> J -> I	2 + 1 + 5 + 3 + 4 + 5 = 22

A* but the heuristic for D is 8 instead of 4

	F(n) = g(n) + h(n)	

A		Explored = { } Frontier = { A }
A -> B A -> C A -> D	2 + 3 = 5 1 + 5 = 6 3 + 8 = 11	min (5, 6, 11) = 5 Explored = { A , B } Frontier = { C, D }
A -> B -> E A -> B -> F	(2 + 7)+ 0 = 9 (2 + 4) + 3 = 9	min (6, 11, 9, 9) = 6 Explored = { A , B, C } Frontier = { D, E, F }
A -> C -> G A -> C -> H	(1 + 5) + 0 = 6 (1 + 5) + 3 = 9	min (11, 9, 9, 6, 9) = 6 Explored = { A , B, C, G } Frontier = { D, E, F , H}
		min (11, 9, 9, 9) = 9 Explored = { A , B, C, G, E } Frontier = { D, F , H}
F -> K	(2+4+ 4)+ 0 = 10	min (11, 9, 9, 10) = 9 Explored = { A , B, C, G, E, F } Frontier = { D, H, K }
		min (11, 9, 10) = 9 Explored = { A , B, C, G, E , F, H} Frontier = { D, K }
		min (11, 10) = 10 Explored = { A , B, C, G, E , F, H, K} Frontier = { D }
A -> D -> I A -> D -> J	(3 + 5) + 0 = 8 (3 + 4) + 0 = 7	Explored = { A , B, C, G, E , F, H, K, D} Frontier = { I, J}
		min(8,7) = 7 Explored = { A , B, C, G, E , F, H, K, D, J} Frontier = { I }
		Explored = { A , B, C, G, E , F, H, K, D, J, I} Frontier = { }

Goal Node	Order of Iteration	Cost Associated
E	A -> B -> C -> G -> E	2 + 1 + 5 +7 = 15
K	A -> B -> C -> G -> E -> F -> H -> K	2 + 1 + 5 + 7 + 4 + 5 + 4 = 28
I	A -> B -> C -> G -> E -> F -> H -> K -> D -> J -> I	2 + 1 + 5 + 7 + 4 + 5 + 4 + 3 + 4 + 5 = 40

2 STRIPS (20 Points)

In a rather unorthodox experiment, Prof. Dr. Darksome created raptors which become more intelligent when they drink alcohol. However, as expected by everyone who has ever seen Jurassic Park, two raptors, hans and klaus, managed to free themselves. The professor is currently hiding in a small closet. The door of the closet d, is currently closed. However, if a raptor is drunk, he is intelligent enough to open a door. The goal of the two raptors is to open the door. hans is at position a, klaus at position b. Both are standing on the floor, making their height normal. A bottle of whisky, talisker, is on a shelf at at position e-the bottle therefore has a height of high. The door to the closet d is at position d. When raptors drink alcohol, they will drink the entire bottle and then drop it. Therefore, we're ignoring the actual drinking in our formalization. However, a raptor becomes drunk if he drinks alcohol, To reach the shelf, hans and klaus must work together. If one of them bows down, the other one can jump on his back, thereby reaching the bottle on the shelf.

The following predicates are given:

raptor (X): X is a raptor.

whisky (X): X is a whisky.

door (D) D is a door

open(X): X is open.

closed (X): X is closed.

drunk(X): X is drunk.

.height(X, Y): X has height Y (either normal, low or high).

pos(X,Y): X is at position Y (either a, b, c or d).

free(X): Raptor X is free, if no other raptor is standing on his shoulders.

on(X,Y): Raptor X is standing on raptor Y's shoulders.

Your tasks are:

1. Write down the initial STRIPS database and the goal state

2. Describe the following actions in STRIPS notation. Please note that some precon- ditions or effects of actions necessary to achieve a consistent formalization may have been left implicit

- **move(R. Xold, Xnew)**: Raptor R moves from position Xold to position Xnew
 This is only possible if he has a height of normal.

- **openDoor(R, D):** Raptor R opens the door D. The raptor must be drunk and at the same position as the door.

- **drink (R, W):** Raptor R drinks whisky W. This is only possible if the raptor and the whisky are at the same positions and have the same heights.

- **bowDown(R):** Raptor R bows down to prepare for some other raptor jumpiing on his back. For this, the raptor must have a height of normal. As a result,the new height is low.

- **straightenUp(R):** A raptor R can straighten up if his height, is low and he has no other raptor standing on his shoulders. As a result, the raptor has a height of normal.

- **climbOn(R1, R2):** Raptor R, climbs upon raptor R2. For this, both must be at the same position. Additionally, R_2 must have a height of low and no other raptor may be standing on his shoulders. Ri must have a height of normal. As a result, R is standing on R_2 and now has a height of high.

- **climbDown(R_1. R) :** Raptor R, climbs down from raptor R. For this, he must be on the other raptor. Afterwards R, has a height of normal again.

Solution :

Initial STRIPS DataBase :

{
raptor(hans),
raptor(klaus),
door(d1),
closed(d1), [or door(d1) ^ closed(d1)]
pos(d1, d), [or door(d1) ^ pos(d1, d)]
pos(hans, a), [or raptor(hans) ^ pos(hans, a)]
pos(klaus, b), [or raptor(klaus) ^ pos(klaus, b)]
height(hans, normal) [or raptor(hans) ^ height(hans, normal)]
height(klaus, normal) [or raptor(klaus) ^ height(klaus, norma)]
free(hans), // hans is standing on the floor and no raptor on shoulder
free(klaus), // klaus is standing on the floor and no raptor on shoulder
whiskey(talisker),
pos(talisker, c), [or whiskey(talisker) ^ pos(talisker, c)]
height(talisker, high) [or whiskey(talisker) ^ height(talisker, high)]
}

Goal : open(d1)

STRIPS NOTATION :

C = Pre-conditions
D = Effect
A = Action

- **move(R, Xold, Xnew)**:

  C : raptor(R), pos(R, Xold), height(R, normal),
  D : pos(R, Xold)
  A : pos(R, Xnew)

- **openDoor(R, D):**

  C : raptor(R), drunk(R), [∃x : pos(R, X) ∧ pos(D, X)],
  D : open(D)

A : closed(D)

- **drink (R, W):**

  C : raptor(R), whiskey(W), [∃x : pos(R, X) ∧ pos(W, X)] , [∃H : height(R, H) ∧ height(W, H)]
  D :
  A : drunk(R)

- **bowDown(R):**

  C : raptor(R), height(R, normal),
  D : height(R, normal)
  A : height(R, low)

- **straightenUp(R):**

  C : raptor(R), height(R, low), free(R)
  D : height(R, low)
  A : height(R, normal)

- **climbOn(R1, R2):**

  C : raptor(R1), raptor(R2), [∃x : pos(R1, X) ∧ pos(R2, X)], height(R2, low) , free(R2), height(R1, normal)
  D : free(R2), height(R1, normal)
  A : height(R1, high), on(R1, R2)

- **climbDown(R1, R2) :**

  C : raptor(R1), raptor(R2), on(R1, R2), height(R1, high)
  D : height(R1, high), on(R1, R2)
  A : height(R1, normal), free(R2)